

Design and Evaluation of a Drift-Aware Continuous Learning MLOps Framework for Financial Transaction Fraud Detection

by

Mashud Hasan
22299053

Nirnoy Biswas
24241185

Md. Sabbir Hossain
22299038

Al Imran
231011035

Imtiaz Zaman Sami
23101551

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
[Month Year]

© 2026. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Mashud Hasan
22299053

Nirnoy Biswas
24241185

Md. Sabbir Hossain
22299038

Al Imran
231011035

Imtiaz Zaman Sami
23101551

Approval

The thesis titled “Design and Evaluation of a Drift-Aware Continuous Learning MLOps Framework for Financial Transaction Fraud Detection” submitted by

1. Mashud Hasan (22299053)
2. Nirnoy Biswas (24241185)
3. Md. Sabbir Hossain (22299038)
4. Al Imran (231011035)
5. Imtiaz Zaman Sami (23101551)

of Spring, 2026 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science and Engineering in Summer 2026.

Examining Committee:

Supervisor:

(Member)

Md. Sabbir Ahmed
Senior Lecturer
Department of Computer Science and Engineering
Brac University

Co-Supervisor:

(Member)

Naima Tahsin Nodi
Adjunct Lecturer
Department of Computer Science and Engineering
Brac University

Thesis Coordinator:

(Member)

Dr. Md. Golam Rabiul Alam
Professor
Department of Computer Science and Engineering
Brac University

Head of Department:

(Chair)

Dr. Sadia Hamid Kazi
Chairperson
Department of Computer Science and Engineering
Brac University

Abstract

Machine-learning models for financial transaction fraud detection are usually trained once and evaluated on a random split of historical data. In production, this picture breaks down in three ways. Fraud patterns and customer behaviour change over time (concept drift), so a model that scores well at deployment gradually loses accuracy. The true label of a transaction only becomes known weeks later, when a chargeback is filed or the dispute window closes (label delay), so the system cannot see its own mistakes in time. And the tools that Machine Learning Operations (MLOps) platforms offer to decide when to retrain, mostly statistical drift detectors on the input data, have rarely been tested against these conditions in fraud detection.

The purpose of this thesis is to build a drift-aware MLOps framework for financial fraud detection that can identify when a deployed model is becoming outdated, decide when retraining is actually necessary, and safely deploy a better retrained model without unnecessary retraining. The framework monitors a deployed LightGBM model on delayed labels, retrains it on a sliding window of recent data, and promotes a new model only after a promotion gate confirms that it is at least as good as the live model on data neither has seen. It is built with MLflow for experiment tracking and model versioning, a FastAPI scoring service and a monitoring dashboard, and is evaluated by replaying the IEEE-CIS Fraud Detection dataset (590,540 transactions over 182 days) as a live stream, with the Sparkov credit-card dataset (1.8 million transactions over two years) as a replication. Retraining strategies are compared under label delays of 0 to 60 days using time-ordered evaluation, paired day-block bootstrap confidence intervals, walk-forward tuning and repeated training seeds.

Preliminary results on IEEE-CIS show that weekly PR-AUC of a model trained once fell from 0.61 to 0.39 while the drift of its scores stayed far below the usual alarm level, that the benefit of retraining shrinks from +0.078 to +0.010 PR-AUC as labels arrive later, and that a fixed retraining schedule on recent data outperformed a tuned drift-triggered policy at every label delay. In fault-injection tests, the promotion gate blocked every model trained on corrupted data. The study therefore argues that a drift-aware fraud system should be designed around what drift detection can and cannot observe under label delay.

Keywords: Financial fraud detection, concept drift, label delay, continuous learning, MLOps, model retraining, promotion gate, LightGBM, PR-AUC, IEEE-CIS

Acknowledgement

Firstly, we thank Allah, the Most High, for everything He has bestowed upon us.

We would like to express our sincere gratitude to our supervisor, Md. Sabbir Ahmed sir, and our co-supervisor, Naima Tahsin Nodi ma'am, for their guidance and support throughout this research. Their feedback helped us to sharpen the research question and to overcome the difficulties we faced during the work.

We also thank the IEEE Computational Intelligence Society and Vesta Corporation for releasing the IEEE-CIS Fraud Detection dataset, the maintainers of the Sparkov credit-card dataset, and all the researchers whose articles and open-source tools have shaped this work.

Table of Contents

Declaration	i
Approval	ii
Abstract	iii
Acknowledgement	iv
List of Figures	vii
List of Tables	viii
Nomenclature	ix
4 Methodology and Design	1
4.1 Design Process or Methodology Overview	1
4.1.1 Research Design	1
4.1.2 Datasets and Preparation	2
4.1.3 Simulated Deployment under Label Delay	4
4.1.4 Retraining Strategies	5
4.1.5 Drift and Performance Measures	6
4.1.6 Statistical Validation	7
4.1.7 Framework Evaluation and Fault Injection	8
4.2 Preliminary Design or Design (Model) Specification	9
4.2.1 Fraud Detection Model	9
4.2.2 Framework Architecture	10
4.2.3 Monitoring on Delayed Labels	12
4.2.4 Retraining Policy and Safety Net	12
4.2.5 Training and Automatic Window Selection	12
4.2.6 Promotion Gate	13

4.2.7 Model Registry, Serving and Dashboard	13
4.2.8 Framework Configuration	14
4.2.9 Implementation and Reproducibility	14
Bibliography	15

List of Figures

4.1 Methodology of the study: from literature review and time-ordered data preparation to the retraining experiments, the framework and its evaluation	1
4.2 Evaluation protocol: each dataset is split in time into a history and a stream that is replayed week by week; settings are tuned walk-forward in 4-week folds	4
4.3 Architecture of the fraud detection model: a transaction's features pass through a sequence of boosted decision trees, whose summed output becomes a fraud probability and, with the validation threshold, a decision	9
4.4 Components of the proposed framework and the flow of data between them	11
4.5 The proposed continuous-learning loop, run every week on the transaction stream .	11

List of Tables

4.1	Datasets used in this study (after preparation)	2
4.2	Retraining strategies compared in the simulation (default settings)	5
4.3	Settings searched by walk-forward tuning (190 configurations)	7
4.4	Faults injected into retraining jobs to test the safeguards	8
4.5	LightGBM settings used for every model in the study	10
4.6	Settings of the framework (configs/default.toml)	14

Nomenclature

The next list describes several symbols and abbreviations that will be used later within the body of the document.

ADWIN	Adaptive Windowing (drift detector)
API	Application Programming Interface
CI	Confidence Interval
DDM	Drift Detection Method
FDS	Fraud Detection System
GNN	Graph Neural Network
KS	Kolmogorov-Smirnov (test)
MLOps	Machine Learning Operations
PR-AUC	Area Under the Precision-Recall Curve
PSI	Population Stability Index
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique

Chapter 4

Methodology and Design

4.1 Design Process or Methodology Overview

This section describes how the study was designed and carried out: the overall research design, the data, the way a deployment is simulated, the retraining strategies that are compared, and how the results and the framework are evaluated. The specification of the model and of the framework itself follows in Section 4.2.

4.1.1 Research Design

This study is an empirical, simulation-based investigation followed by the design and evaluation of a software framework. Chapter 2 showed that the claims made for drift-triggered retraining rest mostly on artificial drift, single datasets and labels that arrive immediately [1], [2], [3]. We therefore first measure, on public fraud data with natural drift, how a deployed model behaves and which retraining policy works best under label delay (research questions RQ1 to RQ3), and only then design the framework around the result and test it (RQ4). The study follows the workflow in Figure 4.1.

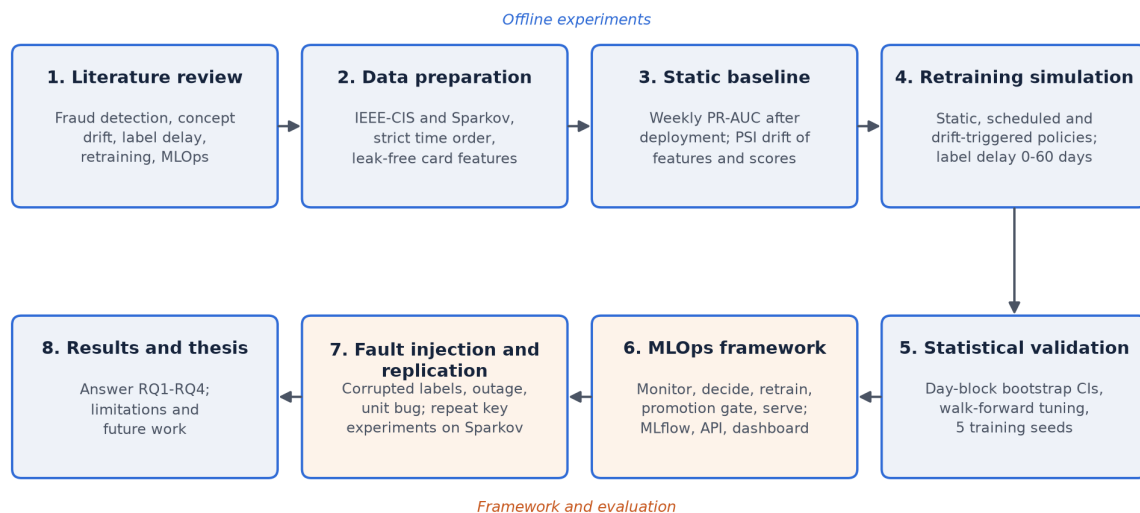


Figure 4.1: Methodology of the study: from literature review and time-ordered data preparation to the retraining experiments, the framework and its evaluation

Three principles guide every experiment. First, **time is respected**: a model is only trained on transactions that happened, and whose labels were available, before the period it is evaluated on [4]. Second, **the deployment is simulated, not imagined**: each dataset is replayed as a stream, week by week, and the label delay is applied explicitly, as described

by Dal Pozzolo et al. [5]. Third, **differences must survive uncertainty**: every comparison is reported with a confidence interval, the settings of competing strategies are tuned with the same procedure, and the key results are repeated with several training seeds and on a second dataset.

The rest of this section describes the datasets and their preparation (Section 4.1.2), the simulated deployment under label delay (Section 4.1.3), the retraining strategies (Section 4.1.4), the drift and performance measures (Section 4.1.5), the statistical validation (Section 4.1.6) and the evaluation of the framework, including fault injection (Section 4.1.7). Section 4.2 then specifies the design: the fraud detection model and each component of the framework.

4.1.2 Datasets and Preparation

Public fraud datasets with reliable timestamps are scarce. Many studies use the European credit-card dataset, which covers only two days and therefore cannot show drift [6], [7]. We use three datasets that cover months to years, summarised in Table 4.1.

Table 4.1: Datasets used in this study (after preparation)

	IEEE-CIS	Sparkov	BAF (Base)
Domain	Real e-commerce card transactions (Vesta)	Simulated card transactions (Sparkov generator)	Bank account opening applications (privacy-preserving synthetic data)
Records	590,540	1,801,969	1,000,000
Time span	182 days	720 days (to 21 Dec 2020)	8 months
Fraud rate	3.50%	0.53%	1.10%
Features used	431 (transaction and identity tables, mostly anonymised)	16 (amount, time, location, customer and card-history features)	29 (applicant, request, device and velocity features)
Time resolution	Seconds from a reference point	Exact timestamp	Month only (time within the month assigned at random)
Stream starts at	60% of rows (day 101)	50% of rows (about 1 Jan 2020)	50% of rows (month 4)
Role	Main dataset	Replication	Second replication

IEEE-CIS Fraud Detection is the main dataset. It was released by the IEEE Computational Intelligence Society and Vesta Corporation and contains 590,540 real e-commerce transactions over 182 days, of which 3.5% are fraudulent. It has been used in recent studies of drift and fraud detection [4], [8]. The transaction and identity tables are joined on the transaction identifier, giving 431 features, most of them anonymised by the provider. Only the time column (seconds from a reference point), the identifier and the label are excluded. Text columns are treated as categorical features, which LightGBM

handles natively, and missing values are kept as missing rather than imputed. More than half of the columns are missing for over 50% of transactions.

Sparkov is the first replication dataset. It contains card transactions for about 1,000 customers produced by the Sparkov transaction simulator between January 2019 and December 2020. After inspection we cut the data at 21 December 2020, because the simulator produces almost no fraud after that date (17 frauds and then none, while the volume doubles), which would make the last weeks meaningless for evaluation. The remaining 1,801,969 transactions cover 720 days with a fraud rate of 0.53%. Names, street addresses, card numbers and transaction numbers are removed. We derive the transaction hour and weekday, the customer's age, and the distance between customer and merchant, and five per-card behaviour features: the time since the card's previous transaction, its number and total amount of transactions in the previous 24 hours, its number of transactions in the previous 30 days, and the amount relative to the card's running average. Each of these is computed only from the card's *earlier* transactions, so no information from the future leaks into a feature. A lifetime transaction count was deliberately not used: it only grows over time and would drift by construction.

Bank Account Fraud (BAF) is the second replication dataset. It is a privacy-preserving synthetic dataset of one million bank account applications over eight months, generated from real data and designed to contain realistic drift between months. We use the Base variant. Values that the providers document as missing (negative values in six columns) are converted to missing values, a constant column is dropped, and the month itself is not used as a feature, so that the model cannot memorise the period. BAF records only the month of each application, so each record is given a random time within its month with a fixed seed. Drift therefore appears between months, while the weeks within a month are exchangeable. This is a limitation of the dataset, and the BAF results are interpreted accordingly.

All three datasets are sorted by time and never shuffled. Each is split at a fixed fraction of its rows into a *history*, used to train the first model, and a *stream*, which is replayed as if it were arriving live (Figure 4.2). The stream starts at 60% of the rows for IEEE-CIS (day 101), and at 50% for Sparkov and BAF, whose longer histories allow a full year or four months of training data before deployment.

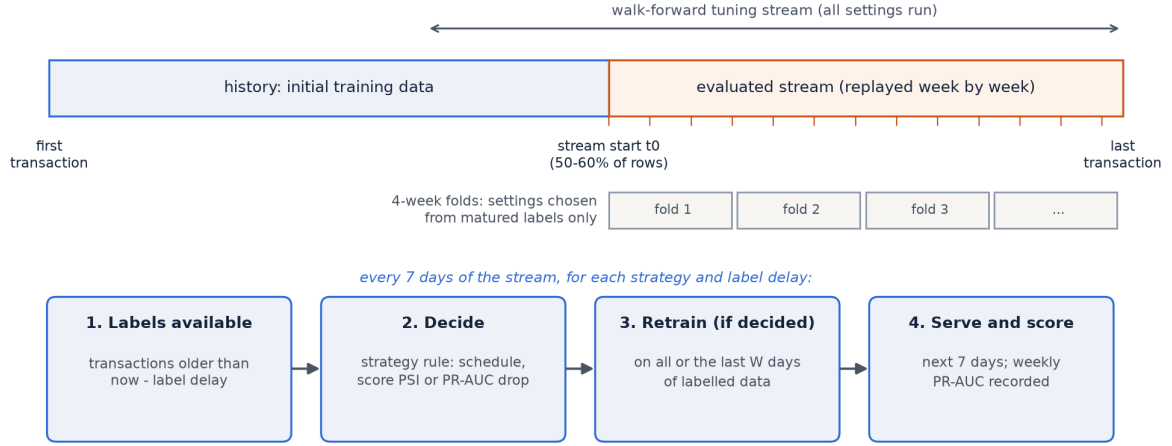


Figure 4.2: Evaluation protocol: each dataset is split in time into a history and a stream that is replayed week by week; settings are tuned walk-forward in 4-week folds

4.1.3 Simulated Deployment under Label Delay

The stream is replayed in steps of seven days, from the stream start to the end of the data. At each step, three things happen in order: the strategy checks which labels are available and decides whether to retrain; if it retrains, a new model is trained and immediately replaces the old one; and the current model scores the transactions of the next seven days. The scores of each week are kept, together with the version of the model that produced them. In production the transactions would arrive through a streaming platform such as Kafka or Flink, as in the real-time pipelines of Kaushik et al. [9]; replaying recorded data instead makes every experiment exactly repeatable.

Label delay is modelled as a fixed number of days d between a transaction and the moment its label can be used. At time t , the labelled data available for monitoring and training is

$$L(t) = \{ x_j : t_j \leq t - d \} \quad (4.1)$$

where t_j is the time of transaction x_j . The first model is trained at the stream start under the same rule, so it never sees labels it could not have had. We evaluate delays of 0, 15, 30 and 60 days. A delay of 0 corresponds to the assumption made by most retraining studies [3], [2]; 30 to 60 days corresponds to the time a cardholder typically needs to notice a fraudulent charge and file a dispute [5]. Investigator feedback on a few alerted transactions, which arrives faster in a real system [5], is not modelled: every label arrives after the same delay. This makes the setting slightly pessimistic for performance-based triggers, a point we return to in the discussion.

4.1.4 Retraining Strategies

Five strategies are compared, covering the options described in Section 2.1 [10], [5]. They are summarised in Table 4.2.

Table 4.2: Retraining strategies compared in the simulation (default settings)

Strategy	When it retrains	Training data	Needs labels to decide?
Static	Never	History before the stream	-
Periodic expanding	Every 14 days	All labelled data so far	No
Periodic sliding	Every 14 days	Labelled data of the last 60 days	No
Drift: score PSI	PSI of the model's scores over the last week, against its validation scores, exceeds 0.1 (at least 14 days apart)	All labelled data so far	No
Drift: performance	PR-AUC on newly matured labels falls more than 15% below the reference (Equation 4.2; at least 14 days apart)	All labelled data so far (or the last 60 days in tuned variants)	Yes (delayed)

The **static** model is the reference: it is trained once at the stream start and never updated. The two **periodic** strategies retrain on a fixed schedule and differ only in their training data: the expanding window uses all labelled data so far, while the sliding window keeps only the most recent 60 days and forgets older patterns.

The **score-PSI** strategy is a typical label-free drift trigger [1], [2]. It compares the distribution of the scores the model produced over the last week with the scores it produced on its own validation data, using the PSI defined in Section 4.1.5, and retrains when the PSI exceeds 0.1, the usual boundary between a stable and a shifted population. It can react immediately because it needs no labels.

The **performance** strategy is an error-based trigger in the spirit of DDM and ADWIN [3], [11], adapted to delayed labels. At each step it computes the model's PR-AUC on transactions whose labels matured during the last 14 days and that the model was not trained on, provided they contain at least 30 frauds. It retrains when

$$AP_{\text{live}}(t) < (1 - \tau) AP_{\text{ref}} \quad (4.2)$$

where $AP_{\text{live}}(t)$ is that live PR-AUC, τ is the tolerance (0.15 by default), and AP_{ref} is either the model's validation PR-AUC or its first live measurement. Both drift strategies wait at least 14 days between retrains, so that one drift episode does not trigger a series of retrains.

Each strategy is run at each label delay, with exactly the same data, model settings and random seed, so that the only difference between runs is the retraining decision and the training window. Models trained on identical windows are cached and reused across

strategies, which makes the large number of runs feasible and guarantees that identical decisions give identical models.

4.1.5 Drift and Performance Measures

Population Stability Index. Drift in a feature or in the model's scores is measured with the PSI, which compares a current distribution with a reference distribution [2]:

$$\text{PSI} = \sum_{i=1}^B (c_i - r_i) \ln(c_i / r_i) \quad (4.3)$$

where the reference values are divided into $B = 10$ bins at their deciles, r_i and c_i are the shares of reference and current values in bin i , missing values form a bin of their own, and shares are floored at 10^{-6} to avoid division by zero. A PSI below 0.1 is usually read as stable, 0.1 to 0.25 as a moderate shift and above 0.25 as a major shift. For the baseline analysis, the PSI is computed for the ten most important features of the static model (by total gain) and for its scores, in every time bin against the training period.

PR-AUC. Fraud is rare and investigators can only review the highest-scored alerts, so the main measure is the area under the precision-recall curve, computed as average precision [5]:

$$\text{AP} = \sum_n (R_n - R_{n-1}) P_n \quad (4.4)$$

where P_n and R_n are the precision and recall at the n -th threshold of the ranked scores. Unlike accuracy, PR-AUC is not inflated by the large number of legitimate transactions, and unlike ROC-AUC it focuses on how clean the top of the alert list is [12]. A model with no skill has a PR-AUC equal to the fraud rate.

Mean weekly PR-AUC. In the stream, different weeks are scored by different model versions, whose scores are not calibrated to each other. Computing a single PR-AUC over all weeks would mix these scales and penalise strategies that retrain often, even when every one of their models ranks transactions well. We therefore compute PR-AUC separately for each week w of the stream and report the mean over the W weeks:

$$\text{mean AP} = (1/W) \sum_{w=1}^W \text{AP}_w \quad (4.5)$$

Weeks without any fraud are skipped. The lowest weekly PR-AUC, the number of retrains and the F1-score at each model's own threshold are reported as secondary measures.

4.1.6 Statistical Validation

Paired day-block bootstrap. Weekly PR-AUC is noisy, especially when a week contains only a few hundred frauds, and consecutive transactions are correlated. Confidence intervals are therefore computed with a bootstrap that resamples whole days rather than single transactions. Within every week, the days of that week are drawn with replacement, and the mean weekly PR-AUC is recomputed. The same resampled days are used for every strategy, so differences between strategies are paired and much of the shared noise cancels out. We use 500 resamples and report 95% percentile intervals. A difference is considered reliable when its interval excludes zero.

Walk-forward tuning. A fair comparison requires that every strategy has good settings, chosen without looking at the data it is evaluated on. All 190 configurations in Table 4.3 are run over a tuning stream that starts at 35% of the rows, earlier than the evaluated stream. The evaluated stream is divided into folds of four weeks. At the start of each fold, each family (static, schedule, drift performance) selects the configuration with the highest score

$$J(c) = \text{mean AP}(c) - \kappa \times N_{\text{retrain}}(c) / (\text{number of 4-week periods}) \quad (4.6)$$

computed only on weeks whose labels have matured by the start of the fold, that is, on information that would really have been available. $\kappa = 0.002$ is a small cost per retrain that breaks ties in favour of simpler configurations. The selected configuration's results on the fold are then recorded, and the next fold repeats the selection. The result is an honest estimate of how each family would perform if an operator tuned it with the information available at the time.

Table 4.3: Settings searched by walk-forward tuning (190 configurations)

Family	Settings searched	Configurations
Static	-	1
Schedule	Expanding window, period 7, 14 or 28 days; sliding window, period 7, 14 or 28 days and window 30 or 60 days	9
Drift: performance	Tolerance 0.05, 0.1, 0.15, 0.2 or 0.3; monitoring window 7, 14 or 28 days; minimum gap 7, 14 or 28 days; reference = validation PR-AUC or first live measurement; training data = all or last 60 days	180

Training seeds. The bootstrap only covers the randomness of the evaluation sample. LightGBM also has randomness in training, through row and feature subsampling. The key configurations are therefore re-run with five random seeds (42, 1, 2, 3 and 4) at label delays of 0 and 30 days, and for each comparison we report the mean and spread of the difference and the number of seeds in which its sign holds.

Replication. Finally, the whole sequence, from the static baseline through the delay sweep, walk-forward tuning, seeds and the framework replay, is repeated unchanged on Sparkov and BAF, with the same scripts and settings. A finding that holds on IEEE-CIS but not on the other datasets is reported as dataset-specific.

4.1.7 Framework Evaluation and Fault Injection

The framework is evaluated by running it over the evaluated stream of each dataset as if it were live, with a label delay of 30 days. The replay uses the same data store, MLflow registry and model code as a deployment would. Its mean weekly PR-AUC is compared with the offline simulation of the same policy, which checks that the framework reproduces the experimental results, and with the static model. To evaluate window selection, four versions of the framework are replayed on each dataset on the same machine: a static model, a 60-day window, all history, and automatic selection. The four replays share one cache of trained models, so a model with the same training data is trained only once and the versions differ only in their decisions.

A retraining pipeline that runs automatically will eventually train on bad data. To test whether the framework's safeguards stop such models, we inject the faults in Table 4.4 into selected retraining jobs of the replay (by default the second and fourth scheduled retrains). The faults imitate common pipeline failures: a misaligned join, an outage of the chargeback feed, and a unit change between training and serving. In the first three, only the data read by the training job is corrupted, while the production feature path and the label store stay correct. The fourth corrupts the label store itself, which the gate also reads, and represents a failure the gate cannot see by design; it tests whether the safety net limits the damage.

Table 4.4: Faults injected into retraining jobs to test the safeguards

Fault	What goes wrong	Real-world cause
Label shuffle	Labels are permuted within the training window, so they no longer match their transactions	A misaligned join in the training job
Label loss	80% of frauds in the training window are recorded as legitimate	An outage of the chargeback feed
Feature unit bug	The five most important numeric features of the live model are multiplied by 100 in training only	A unit change (e.g. cents and dollars) between training and serving
Upstream label shuffle	The label store itself is corrupted, so the promotion gate also reads wrong labels	Corruption at the source; a known blind spot

Each fault scenario is replayed twice, once with the promotion gate and once without it, and compared with a clean replay. A safeguard is considered effective if the run with the fault and the safeguard stays close to the clean run, while the run without the safeguard loses performance. Each scenario uses its own MLflow store, so the main registry is never affected.

4.2 Preliminary Design or Design (Model) Specification

This section specifies the design: the fraud detection model used throughout the study, and the drift-aware continuous learning framework built on the results of the experiments, component by component, with its configuration and implementation.

4.2.1 Fraud Detection Model

All experiments use LightGBM, a gradient boosted decision tree library, with the settings in Table 4.5. Gradient boosted trees are strong and widely used models for tabular fraud data [8], [13], they handle missing values and categorical features natively, and they train fast enough to be retrained hundreds of times. Amekoe et al. also found that batch-trained boosted trees remain competitive with incremental learners when labels arrive late [13], which is exactly the setting of this study. The goal is not to find the best possible classifier, as many static studies do [7], [12], [14], but to keep one strong and realistic model fixed so that differences between retraining strategies can be attributed to the strategies alone. No resampling such as SMOTE is used, because it can distort the evaluation [6]; class imbalance is handled by weighting the fraud class instead, a cost-sensitive approach that Somasundaram and Reddy also used to handle imbalance in a drifting fraud stream [15].

Figure 4.3 shows the architecture of the model. Each transaction is described by its features, 431 on IEEE-CIS, 16 on Sparkov and 29 on BAF. LightGBM builds an ensemble of decision trees one after another, each with up to 256 leaves, and every new tree is fitted to the errors that the trees before it still make. Training stops when the validation AUC has not improved for 100 rounds, at most after 2,000 trees. For a new transaction, the outputs of all trees are summed into a score $F(x)$, which a sigmoid function turns into a fraud probability between 0 and 1; the transaction is flagged as fraud when this probability reaches the decision threshold described below.

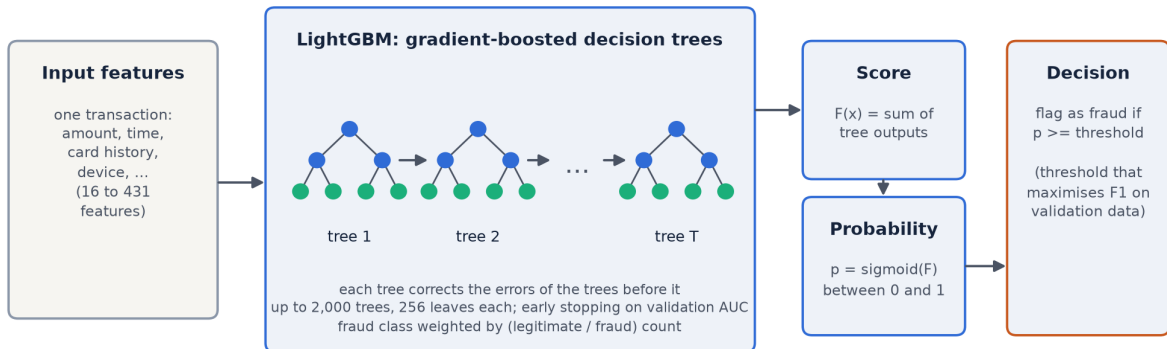


Figure 4.3: Architecture of the fraud detection model: a transaction's features pass through a sequence of boosted decision trees, whose summed output becomes a fraud probability and, with the validation threshold, a decision

Table 4.5: LightGBM settings used for every model in the study

Setting	Value	Purpose
Objective	binary	Fraud probability for each transaction
Learning rate	0.05	Step size of each boosting round
Number of leaves	256	Capacity of each tree
Minimum samples per leaf	100	Prevents leaves fitted to a few transactions
Row subsampling	0.8, every round	Randomness and regularisation
Feature subsampling	0.5 per tree	Randomness and regularisation
L2 regularisation	1.0	Shrinks leaf values
Class weight of fraud	legitimate / fraud count in the training data	Compensates for class imbalance
Boosting rounds	up to 2,000, early stopping after 100 without improvement	Stops on the validation part of the training window
Validation part	most recent 15% of the training window	Early stopping, threshold choice and reference PR-AUC

Every training run uses a time-ordered validation split: the most recent 15% of the training window is held out for early stopping, for choosing the decision threshold, and for recording the model's reference PR-AUC. The decision threshold is the one that maximises the F1-score on this validation part. It is used for the F1-score, precision and recall reported alongside PR-AUC and for the fraud flags returned by the scoring service; PR-AUC itself does not depend on a threshold. Alessi and Fugini manage the threshold dynamically during operation to balance precision and recall [16]; in this design each model keeps the threshold chosen at training time, and a new threshold arrives with every retrained model.

For the static baseline in RQ1, the first 50% of each dataset is used for training, the next 10% for validation, and the remaining 40% is divided into ten consecutive time bins, so that performance can be followed over the months after deployment.

4.2.2 Framework Architecture

The framework turns the experimental findings into a working MLOps system. Its components are shown in Figure 4.4 and its settings in Table 4.6. It runs the loop of Figure 4.5 every seven days: monitor, decide, retrain, check, and serve. Like the closed-loop MLOps framework of Reda et al. for phishing detection [17], it connects monitoring, retraining and deployment in one automated loop; unlike it, its decisions are designed for weeks of label delay rather than near-immediate feedback, so retraining follows a schedule instead of being driven by drift events.

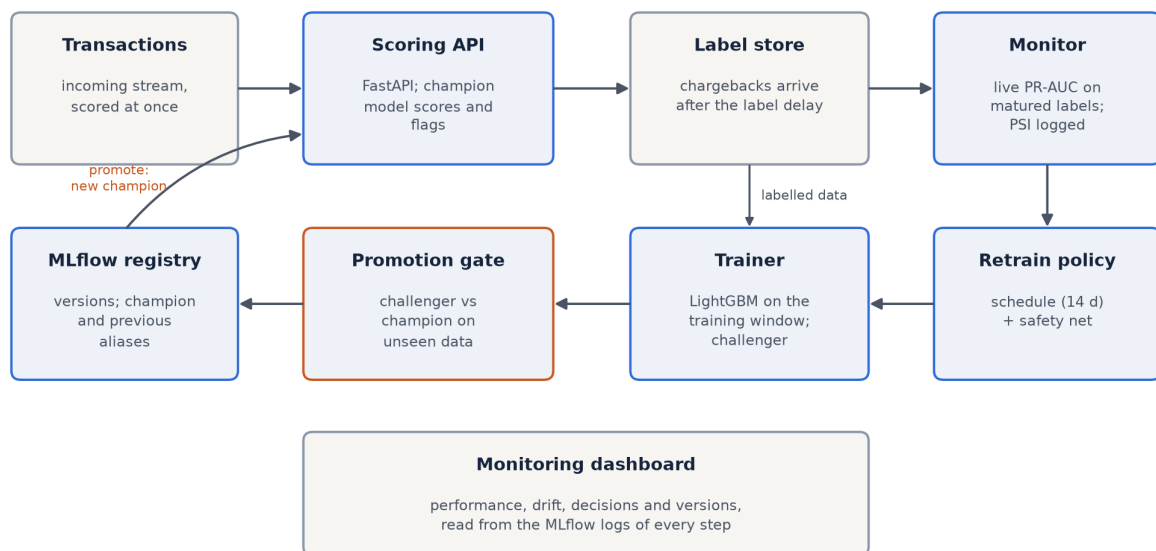


Figure 4.4: Components of the proposed framework and the flow of data between them

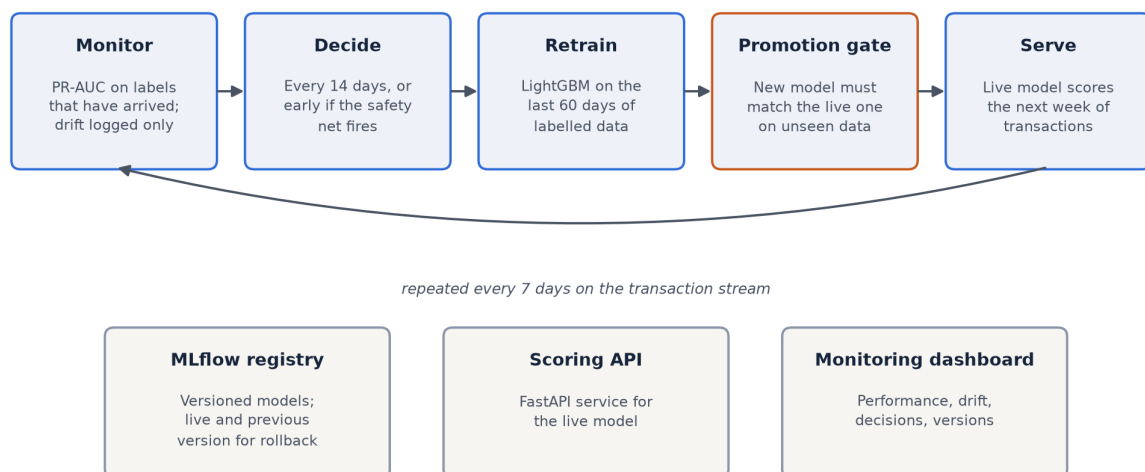


Figure 4.5: The proposed continuous-learning loop, run every week on the transaction stream

4.2.3 Monitoring on Delayed Labels

The monitor measures the live model's PR-AUC on the labels that matured during the last 14 days, restricted to transactions the model was not trained on. It also computes the PSI of the model's scores and of its ten most important numeric features against a sample of 50,000 rows of its training window, and logs both to MLflow. In line with the experimental results, the PSI is logged for diagnosis only and never triggers retraining on its own. Two forms of monitoring are deliberately left outside the current design: tracking drift separately for each source of data, as DriftGuard-TriAudit does for the numbers, text and layout of financial statements [18], and tracking the stability of model explanations, which John showed can become unstable and unfair as populations drift [19]. Both are natural extensions once the framework is used with several data sources or with explanations shown to investigators.

4.2.4 Retraining Policy and Safety Net

The primary policy is a fixed schedule: a new model is trained every 14 days on the last 60 days of labelled data. The schedule is complemented by a *safety net* that retrains early, at most once every seven days, when performance collapses between scheduled retrains:

$$AP_{\text{live}} < (1 - \tau) \max(AP_{\text{val}}, \text{median}_4(AP_{\text{live}})) \text{ or } AP_{\text{live}} < \lambda \pi \quad (4.7)$$

with $\tau = 0.30$, $\lambda = 3$ and π the fraud rate among the matured labels. The first condition compares live PR-AUC with the better of the model's own validation PR-AUC and the median of the last four live measurements. Using recent live history as well as the validation score matters: a model trained on corrupted data can report a poor validation score, which would otherwise lower the bar it is measured against. The second condition is a no-skill check, since a model without skill has a PR-AUC close to the fraud rate.

4.2.5 Training and Automatic Window Selection

The trainer fits a LightGBM model with the settings of Table 4.5 on the policy's training window, which ends one label delay before the present. The window length can be set to zero for an expanding window. Each new model is registered in MLflow as a *challenger*, together with its training window, validation PR-AUC and threshold.

Automatic window selection. The experiments showed that the best training window depends on the dataset: the last 60 days on IEEE-CIS, but all labelled history on Sparkov and BAF. A fixed window chosen on one dataset can be wrong for another. The framework can therefore select the window itself. At every retrain it trains one challenger m_w per candidate window w , here the last 60 days and all labelled history, and scores all of them on the same evaluation rows E : the most recent labelled transactions that none of the candidates was trained on and the champion was not trained on either. It keeps

$$w^* = \operatorname{argmax}_{w \in W} \operatorname{AP}_E(m_w), \quad W = \{60 \text{ days, all history}\} \quad (4.8)$$

and only this challenger goes on to the promotion gate; the others are registered as not selected. The choice uses only data that would be available at the time, so it needs no knowledge of which window suits the dataset. Because the candidates are trained on identical data except for their start date, the extra cost is one additional training run per retrain.

4.2.6 Promotion Gate

A challenger replaces the live model (the *champion*) only if it passes the promotion gate, following the champion-challenger pattern of MLOps practice [10], [20]. The gate compares both models on the challenger's validation data, restricted to transactions that the champion was not trained on, so that neither model is judged on data it has seen. The challenger is promoted if

$$\operatorname{AP}(\text{challenger}) \geq \operatorname{AP}(\text{champion}) - \delta \quad (4.9)$$

with $\delta = 0.01$. Both models are re-scored through the production feature path rather than with scores stored at training time. This detail is essential: a model trained on wrongly scaled features looks good on its own, equally wrong, validation data, but fails on the inputs it will actually receive in production. If the comparison data contains fewer than 30 frauds, the comparison is not reliable and the challenger is promoted by default.

4.2.7 Model Registry, Serving and Dashboard

MLflow stores every model version with its parameters, metrics and training window. The model in service carries the alias *champion* and its predecessor the alias *previous*, so a rollback is a single alias change. A FastAPI service loads the champion and offers three endpoints: */health* reports the version being served, */predict* returns a fraud probability and flag for each submitted transaction, and */reload* switches to a newly promoted champion without restarting. A monitoring dashboard, generated from the MLflow logs, shows weekly performance, drift, every retraining decision with its reason, and the history of model versions.

4.2.8 Framework Configuration

Table 4.6: Settings of the framework (configs/default.toml)

Setting	Value	Meaning
Loop interval	7 days	How often the framework monitors, decides and serves
Label delay d	30 days	Labels usable this long after the transaction
Retraining schedule	14 days	Primary policy
Training window	60 days	Sliding window of labelled data (0 = expanding)
Candidate windows	60 days, all	With automatic selection: one challenger per window, the best one is gated
Monitoring window	14 days	Matured labels used for live PR-AUC
Minimum frauds	30	Fewer frauds: live PR-AUC is not computed
Safety-net tolerance τ	0.30	Relative drop that triggers an early retrain
History for the reference	4	Recent live measurements in the median
No-skill factor λ	3	Live PR-AUC below 3 x fraud rate triggers
Minimum gap between retrains	7 days	Prevents repeated early retrains
Gate tolerance δ	0.01	Largest PR-AUC loss a challenger may show
Gate minimum frauds	30	Fewer frauds: the challenger is promoted by default
Drift features tracked	10	Most important numeric features (PSI logged)

4.2.9 Implementation and Reproducibility

The study is implemented in Python with LightGBM, pandas, NumPy and scikit-learn for the experiments, MLflow with an SQLite backend for tracking and the model registry, and FastAPI for the scoring service. The framework is organised as a Python package with separate modules for configuration, data access, training, monitoring, the retraining policy, the promotion gate, the registry, fault injection, serving and the dashboard, and is configured with one TOML file per dataset. Twenty-four automated tests on small synthetic data check the components and the full loop.

The long experiments, which take several hours each, save their results after every completed label delay or seed and can resume after an interruption such as a power cut. One runner script reproduces the whole replication on any of the datasets, and Kaggle notebooks run it, and the window-selection comparison, on Kaggle's CPU servers. The experiments were run on a desktop computer and on Kaggle notebooks with four CPU cores. All code, configurations and instructions are kept in a public Git repository, so that every number in this thesis can be regenerated from the public datasets.

Bibliography

- [1] M. N. P. Shakil, M. S. Islam, N. Noman, and M. Papini, "Feature drift-guided adaptive ML retraining: An MLOps approach for big data analytics," in *2025 IEEE International Conference on Big Data (BigData)*, Macau, China, 2025, pp. 7719-7727. doi: 10.1109/BigData66926.2025.11401445.
- [2] H. M. Wong and S. Perumal, "AI-driven model-retraining architecture to sustain operational accuracy in data-drifting environments," in *2025 IEEE Symposium on Wireless Technology & Applications (ISWTA)*, Penang, Malaysia, 2025, pp. 1-6. doi: 10.1109/ISWTA68114.2025.11329530.
- [3] V. Yelleti, "ROSF: Robust online streaming fraud detection with resilience to concept drift in data streams," arXiv preprint arXiv:2504.10229, 2025.
- [4] R. S. Menezes and R. H. Filho, "Semantic and structural drift in financial knowledge graphs: A robustness analysis of GNN-based fraud detectors," in *2025 IEEE International Conference on Knowledge Graph (ICKG)*, Limassol, Cyprus, 2025, pp. 285-291. doi: 10.1109/ICKG66886.2025.00044.
- [5] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3784-3797, 2018. doi: 10.1109/TNNLS.2017.2736643.
- [6] T. Dang, T. Tran, L. Tuan, and M. Tiep, "Machine learning based on resampling approaches and deep reinforcement learning for credit card fraud detection systems," *Applied Sciences*, vol. 11, no. 21, p. 10004, 2021. doi: 10.3390/app112110004.
- [7] I. D. Mienye and Y. Sun, "A deep learning ensemble with data resampling for credit card fraud detection," *IEEE Access*, vol. 11, pp. 30628-30638, 2023. doi: 10.1109/ACCESS.2023.3262020.
- [8] M. N. Uddin and M. M. Aziz, "Shapley value-guided adaptive ensemble learning for explainable financial fraud detection with U.S. regulatory compliance validation," arXiv preprint arXiv:2604.14231, 2026.
- [9] S. Kaushik, K. K. Dokka, D. Patel, Z. Mamadiyarov, and B. Matchanova, "Real-time analytics with intelligent data pipelines and ML detection," in *2025 International Conference on Sustainability, Innovation & Technology (ICSIT)*, Nagpur, India, 2025, pp. 1-7. doi: 10.1109/ICSIT65336.2025.11295098.
- [10] M. R. Pulicharla, "Detecting and addressing model drift: Automated monitoring and real-time retraining in ML pipelines," *World Journal of Advanced Research and Reviews*, vol. 3, no. 2, pp. 147-152, 2019. doi: 10.30574/wjarr.2019.3.2.0189.
- [11] K. I. Al-Daoud and I. A. Abu-ALSondos, "Robust AI for financial fraud detection in the GCC: A hybrid framework for imbalance, drift, and adversarial threats," *Journal of Theoretical and Applied Electronic Commerce Research*, vol. 20, no. 2, p. 121, 2025. doi: 10.3390/jtaer20020121.

- [12] A. Abd El-Naby, E. E.-D. Hemdan, and A. El-Sayed, "An efficient fraud detection framework with credit card imbalanced data in financial services," *Multimedia Tools and Applications*, vol. 82, pp. 4139-4160, 2023. doi: 10.1007/s11042-022-13434-6.
- [13] K. M. Amekoe, M. Lebbah, G. Jaffre, H. Azzag, and Z. Chelly Dagdia, "Evaluating the efficacy of instance incremental vs. batch learning in delayed label environments: An empirical study on tabular data streaming for fraud detection," arXiv preprint arXiv:2409.10111, 2024. doi: 10.48550/arXiv.2409.10111.
- [14] I. Sohony, R. Pratap, and U. Nambiar, "Ensemble learning for credit card fraud detection," in *Proceedings of the ACM India Joint International Conference on Data Science and Management of Data (CoDS-COMAD)*, 2018, pp. 289-294. doi: 10.1145/3152494.3156815.
- [15] A. Somasundaram and S. Reddy, "Parallel and incremental credit card fraud detection model to handle concept drift and data imbalance," *Neural Computing and Applications*, vol. 31, pp. 3-14, 2019. doi: 10.1007/s00521-018-3633-8.
- [16] G. Alessi and M. Fugini, "Adaptive real-time financial fraud detection with explainable AI tools," *Digital Threats: Research and Practice*, vol. 7, no. 1, pp. 1-31, 2026. doi: 10.1145/3794859.
- [17] A. Reda, S. A. Taie, and M. E. Shaheen, "Hybrid MLOps framework for automated lifecycle management of adaptive phishing detection models," *Scientific Reports*, vol. 15, no. 1, p. 38478, 2025. doi: 10.1038/s41598-025-23600-z.
- [18] Y. Hassan, "DriftGuard-TriAudit: Concept-drift-aware continual multimodal learning for evolving financial statement fraud detection," *Journal of Computing and Electronic Information Management*, vol. 22, no. 1, 2026. doi: 10.54097/5bjxgq31.
- [19] S. John, "Fair and explainable credit-scoring under concept drift: Adaptive explanation frameworks for evolving populations," arXiv preprint arXiv:2511.03807, 2025.
- [20] N. Kodakandla, "Data drift detection and mitigation: A comprehensive MLOps approach for real-time systems," *International Journal of Science and Research Archive*, vol. 12, no. 1, pp. 3127-3139, 2024. doi: 10.30574/ijsra.2024.12.1.0724.